Google Engineer Routine Blueprint

A complete 2-year system for an absolute beginner

at 01-edu — by Claude

Sleep · Deep Work · 01-edu · Peer Learning · Roadmap

Start with why. Build with discipline. Ship with intention.

The system at a glance

This blueprint is designed for an absolute beginner admitted to 01-edu, with a clear 2-year target: Google-level engineering readiness. Every hour in this plan is intentional. Nothing is filler.

8 pm – 2 am	Sleep	6 hours, sacred and non-negotiable
2 am – 6 am	Deep work block	DSA, CS theory, build, read
6 am – 9 am	Morning prep	Body, food, commute
9 am – 5 pm	01-edu	Projects, peer learning, code review
5 pm – 8 pm	Wind down	Journal, dinner, no screens

Google Engineer Routine Blueprint

A complete 2-year system for an absolute beginner

at 01-edu — by Claude

Sleep · Deep Work · 01-edu · Peer Learning · Roadmap

Start with why. Build with discipline. Ship with intention.

Morning ritual — 2 am to 7 am

Why 2 am? Because 8 pm to 2 am captures your full deep sleep cycle. Waking at 2 am puts you in the highest-quality thinking window of any 24-hour period. No noise, no notifications, no social pressure — just you and the work.

2:00 am — 15 min

Wake up — no phone, no scroll

Sit up immediately. Drink the water you placed on your desk the night before. Splash cold water on your face. Sit quietly for 5 minutes — no input.

non-negotiable · water first · no phone

2:15 am — 15 min

Body activation

20 push-ups or jumping jacks. Stretch neck and back. Light snack if hungry. You are not working out — you are waking your nervous system.

movement · no gym needed

2:30 am — 30 min

Review — what happened yesterday

Read your 3 journal sentences from last night. Skim yesterday's code. Write one sentence: 'Today I will solve ___'.

journal · no new material yet · consolidation

3:00 am — 90 min

Deep work — DSA or CS theory

Your sharpest window. One problem or one concept only. If stuck 20+ min, write out your confusion before looking it up. After solving, explain the time complexity aloud.

LeetCode · phone off · no multitasking

4:30 am — 60 min

Build or implement — hands-on output

Implement the concept from scratch, no reference. Push to GitHub even if it is 20 lines. Write a 3-line comment: what it does and why.

GitHub commit · output-based · build something real

5:30 am — 30 min

Reading — feed the long game

Mon/Wed: Clean Code or The Pragmatic Programmer. Tue/Thu: Google, Uber, or Meta engineering blog. Fri/Sat: Designing Data-Intensive Applications (start month 7).

books · eng blogs · 30 min hard stop

6:00 am — 60 min

Prepare for the day

Shower, dress, eat a proper breakfast. During commute: light podcast or silence — no more input. Arrive at 9 am ready to contribute, not just attend.

breakfast · commute · no cramming

01-edu school day — 9 am to 5 pm

01-edu's peer project model is already elite training. The morning deep work block is a force multiplier on top of it. Show up with intent every single day.

9:00 am — 15 min

Arrive — set your intention

Before opening anything, write one sentence on paper: what you will finish today. Not what you'll try — what you will finish.

paper only · one clear target

9:15 am — 105 min

Deep project block — solo, heads down

Read the spec fully first. Plan on paper — draw the logic and data flow. Code with the spec open. If stuck 20+ min, write out your confusion before asking a peer.

no interruptions · plan before code

11:00 am — 60 min

Peer collaboration — structured, not casual

Compare approaches, not answers. Ask 'why did you do it this way?' before suggesting a change. Teach one thing you understood this morning.

compare approaches · teach one thing

12:00 pm — 60 min

Lunch — actually rest, no screens

Eat a real meal. Walk outside if possible — even 10 min of daylight resets your alertness. Talk to peers about anything except the current project.

no laptop · actual rest

1:00 pm — 120 min

Afternoon build — go deeper than required

Finish the spec first — never leave a requirement incomplete. Then ask: what makes this more efficient or readable? Refactor one part. Write a README.

spec first · then go deeper · README always

3:00 pm — 90 min

Code review and peer evaluation

Read their code fully before commenting. Give specific feedback — 'this loop has O(n2) complexity, here is why.' When receiving feedback, write it down, do not argue.

give specific feedback · write it down

4:30 pm — 30 min

Daily close — before you leave

Push everything to GitHub with a clear commit message. Write 3 sentences in your journal: what you built, what confused you, what you tackle tomorrow. Close all tabs.

GitHub push · journal · clear the desk

Weekly focus rotation

Day	Morning focus (2–6 am)	Evening after 01-edu
Monday	DSA — arrays and strings	Review school project
Tuesday	CS theory — OS and memory	Rest, light reading
Wednesday	DSA — trees and recursion	Write about what you built
Thursday	Networking and systems	Peer code review prep
Friday	DSA — hash maps and graphs	Weekly reflection + GitHub
Saturday	Systems design + mock interview	Draw a system on paper
Sunday	Rest + plan next week	Set 3 targets for the week

2-year roadmap

Phase 1 | Months 1–6

Foundation — do not rush this

- → Pick one language and go deep — understand memory, not just syntax
- → Master arrays, linked lists, recursion, sorting, Big O notation
- → Build tiny projects that solve real problems you actually encounter
- → Read Clean Code, start your learning journal on day one

Phase 2 | Months 7–12

Intermediate depth — go harder on DSA

- → Trees, graphs, dynamic programming, hash maps — solve 100+ LeetCode mediums
- → Read Designing Data-Intensive Applications (first half)
- → Learn how databases, caches, and load balancers work at a high level
- → Make your first open source contribution, even a small one

Phase 3 | Months 13–18

Projects that matter — build things you can defend

- → 2–3 substantial projects each solving a real problem with clear tradeoffs
- → Document every decision: why did you make this architectural choice?
- → Start mock interviews once a week minimum with a peer
- → Finish DDIA, begin system design practice seriously

Phase 4 | Months 19–24

Interview-ready — polish, simulate, ship

- → Weekly timed mock interviews — with a peer or on Pramp / interviewing.io
- → GitHub profile tells a clear story: clean READMEs, consistent commits
- → 200+ LeetCode solved (mix of medium and hard), system design fluency
- → Apply to pre-Google companies first to calibrate and sharpen your skills

Non-negotiable daily habits

These six habits are the difference between someone who completes a program and someone who lands the job. Do them every day without exception.

Write daily	5 sentences every night. What you learned, where you got stuck, what is next. This is your flight recorder.
Push to GitHub daily	Even a small commit. Your contribution graph is your proof of work. It compounds visibly over two years.
Teach one thing daily	Explain one concept to a peer. Teaching exposes gaps in your understanding faster than any test.
Read 30 min nightly	Technical books or engineering blogs only. Compound knowledge over 2 years is enormous.
Bed at 8 pm always	The early wake only works if the bedtime is sacred. No exceptions on weekdays, ever.
Weekly review Sunday	3 wins, 1 failure, 3 targets for next week. Adjust the plan, not the goal.

The plan is solid. Execution is the only variable now.

Start tonight.